

Core Language Working Group "ready" Issues for the June, 2021 meeting

References in this document reflect the section and paragraph numbering of document [WG21 N4878](#).

2397. auto specifier for pointers and references to arrays

Section: 9.3.4.5 [dcl.array] **Status:** ready **Submitter:** Hubert Tong **Date:** 2019-02-04

According to 9.3.4.5 [dcl.array] paragraph 1,

In a declaration τ D where D has the form

$D1$ [*constant-expression*_{opt}] *attribute-specifier-seq*_{opt}

and the type of the identifier in the declaration τ D1 is “*derived-declarator-type-list* τ ”, then the type of the identifier of D is an array type; if the type of the identifier of D contains the *auto type-specifier*, the program is ill-formed.

This formulation forbids useful constructs like

```
int a[3];  
auto (*p)[3] = &a;
```

(accepted by current implementations) and should be relaxed to accommodate such cases.

Notes from the February, 2019 meeting:

CWG agreed that the example should be accepted.

Notes from the May 25, 2021 teleconference:

It was observed that CWG rejected the same example as being "not a defect" in considering issue 1222. However, the use of *auto* has significantly expanded since that time and the prohibition of such declarations now seems inconsistent.

Proposed resolution, May, 2021:

1. Change 9.3.4.5 [dcl.array] paragraph 4 as follows:

U is called the array element type; this type shall not be a placeholder type (9.2.9.6 [dcl.spec.auto]), a reference type, a function type, an array of unknown bound, or cv void.

2. Change 9.3.4.6 [dcl.fct] paragraph 11 as follows:

The return type shall be a non-array object type, a reference type, or cv void. **[Note: An array of placeholder type is considered an array type. —end note]**

3. Change 9.2.9.6.2 [dcl.type.auto.deduct] paragraph 2 as follows:

A type τ containing a placeholder type, and a corresponding initializer E , are determined as follows:

- for a non-discarded return statement that occurs in a function declared with a return type that contains a placeholder type, τ is the declared return type and E is the operand of the return statement. If the return statement has no operand, then E is `void()`;
- for a variable declared with a type that contains a placeholder type, τ is the declared type of the variable and E is the initializer. If the initialization is direct-list-initialization, the initializer shall be a *braced-init-list* containing only a single *assignment-expression* and E is the *assignment-expression*;
- for a non-type template parameter declared with a type that contains a placeholder type, τ is the declared type of the non-type template parameter and E is the corresponding template argument.

τ shall not be an array type. In the case of a return statement with no operand...

2448. Cv-qualification of arithmetic types and deprecation of volatile

Section: 6.8.2 [basic.fundamental] **Status:** ready **Submitter:** Alisdair Meredith **Date:** 2020-03-23

According to the definitions in 6.8.2 [basic.fundamental], the arithmetic types include only the non-cv-qualified versions. In the taxonomy of fundamental types, the first mention of “cv-qualified versions of these types” is for scalar types (6.8 [basic.types] paragraph 9). However, 7.6.1.6 [expr.post.incr] paragraph 1 and 7.6.2.3 [expr.pre.incr] paragraph 1 both say:

The type of the operand shall be an arithmetic type other than cv `bool`, or...

which is a contradiction, since cv-qualified `bool` is not an arithmetic type. Similarly, 7.6.19 [expr.ass] paragraph 6 requires an arithmetic type for `+=` and `-=`. D.6 [depr.volatile.type] deprecates the increment and decrement operators when applied to volatile-qualified arithmetic types, but the wording already made those ill-formed (since the normative wording requires an arithmetic type and not a possibly cv-qualified version thereof).

A related question is whether 12.5 [over.built], which explicitly allows for cv-qualified arithmetic types, should also note the deprecation.

See also issue 2185.

Notes from the July, 2020 teleconference:

CWG felt that no changes should be made to 12.5 [over.built].

Proposed resolution (April, 2021):

1. Change 6.8.2 [basic.fundamental] paragraphs 11 and 12 as follows, splitting paragraph 12 as indicated:

Types `bool`, `char`, `wchar_t`, `char8_t`, `char16_t`, `char32_t`, and the signed and unsigned integer types, **and cv-qualified versions (6.8.4 [basic.type.qualifier]) thereof**, are collectively **called termed** *integral types*. A synonym for integral type is *integer type*. [Note 8: Enumerations (9.7.1 [dcl.enum]) are not integral; however, unscoped enumerations can be promoted to integral types as specified in 7.3.7 [conv.prom]. —end note]

~~There are three floating-point types:~~ **The three distinct types** `float`, `double`, and `long double` **can represent floating-point numbers**. The type `double` provides at least as much precision as `float`, and the type `long double` provides at least as much precision as `double`. The set of values of the type `float` is a subset of the set of values of the type `double`; the set of values of the type `double` is a subset of the set of values of the type `long double`. **The types `float`, `double`, and `long double`, and cv-qualified versions (6.8.4 [basic.type.qualifier]) thereof, are collectively termed *floating-point types***. The value representation of floating-point types is implementation-defined. [Note 9: This document imposes no requirements on the accuracy of floating-point operations; see also 17.3 [support.limits]. —end note]

Integral and floating-point types are collectively **called termed** *arithmetic types*. Specializations of the standard library template `std::numeric_limits` (17.3 [support.limits]) shall specify the maximum and minimum values of each arithmetic type for an implementation.

2. Change 6.8.4 [basic.type.qualifier] paragraph 1 as follows, splitting the paragraph as indicated:

~~A type mentioned in 6.8.2 [basic.fundamental] and 6.8.3 [basic.compound] is a cv-unqualified type. Each type which is a cv-unqualified object type or is void (6.8 [basic.types]) has three corresponding cv-qualified versions of its type~~ **other than a function or reference type is part of a group of four distinct, but related, types: a cv-unqualified version, a const-qualified version, a volatile-qualified version, and a const-volatile-qualified version. The type of an object (6.7.2 [intro.object]) includes the cv-qualifiers specified in the decl-specifier-seq (9.2 [dcl.spec]), declarator (9.3 [dcl.decl]), type-id (9.3.2 [dcl.name]), or new-type-id (7.6.2.8 [expr.new]) when the object is created. The types in each such group shall have the same representation and alignment requirements (6.7.6 [basic.align]). [Footnote: The same representation and alignment requirements are meant to imply interchangeability as arguments to functions, return values from functions, and non-static data members of unions. —end footnote] A function or reference type is always cv-unqualified.**

- A *const object* is an object of type `const T` or a non-mutable subobject of a `const` object.
- A *volatile object* is an object of type `volatile T` or a subobject of a `volatile` object.
- A *const volatile object* is an object of type `const volatile T`, a non-mutable subobject of a `const volatile` object, a `const` subobject of a `volatile` object, or a non-mutable `volatile` subobject of a `const` object.

~~The cv-qualified or cv-unqualified versions of a type are distinct types; however, they shall have the same representation and alignment requirements (6.7.6 [basic.align]).⁴⁰~~ **[Note: The type of an object (6.7.2 [intro.object]) includes the cv-qualifiers specified in the decl-specifier-seq (9.2 [dcl.spec]), declarator (9.3 [dcl.decl]), type-id (9.3.2 [dcl.name]), or new-type-id (7.6.2.8 [expr.new]) when the object is created. —end note]**

3. Change 12.5 [over.built] paragraphs 2-10 as follows:

In this subclause, the term *promoted integral type* is used to refer to those **cv-unqualified** integral types which are preserved by integral promotion (7.3.7 [conv.prom]) (including e.g. `int` and `long` but excluding e.g. `char`). [Note 2: In all cases where a promoted integral type is required, an operand of unscoped enumeration type will be acceptable by way of the integral promotions. —end note]

In the remainder of this subclause, *vq* represents either `volatile` or no cv-qualifier.

For every pair (*T*, *vq*), where *T* is ~~an~~ **a cv-unqualified** arithmetic type other than `bool` **or a cv-unqualified pointer to (possibly cv-qualified) object type**, there exist candidate operator functions of the form

```
vq T& operator++(vq T&);  
T operator++(vq T&, int);
```

~~For every pair (*T*, *vq*), where *T* is an arithmetic type other than `bool`, there exist candidate operator functions of the form~~

```
vq T& operator--(vq T&);  
T operator--(vq T&, int);
```

~~For every pair (*T*, *vq*), where *T* is a cv-qualified or cv-unqualified object type, there exist candidate operator functions of the form~~

```
T*vq& operator++(T*vq&);  
T*vq& operator--(T*vq&);  
T*_____operator++(T*vq&, int);  
T*_____operator--(T*vq&, int);
```

For every ~~cv-qualified or cv-unqualified~~ **(possibly cv-qualified)** object type *T* **and for every function type *T* that has neither cv-qualifiers nor a ref-qualifier**, there exist candidate operator functions of the form

```
T& operator*(T*);
```

~~For every function type *T* that does not have cv-qualifiers or a ref-qualifier, there exist candidate operator functions of the form~~

```
T& operator*(T*);
```

For every type *T* there exist candidate operator functions of the form

```
T* operator+(T*);
```

For every **cv-unqualified** floating-point or promoted integral type *T*, there exist candidate operator functions of the form

```
T operator+(T);  
Tl operator-(T);
```

[Drafting note: 20.15 [meta] regarding type traits appropriately handles cv-qualified and cv-unqualified types and does not require revision.]

2458. Value category of expressions denoting non-static member functions

Section: 7.6.1.5 [expr.ref] **Status:** ready **Submitter:** Andrey Erokhin **Date:** 2020-07-04

Expressions denoting non-static member functions are currently classified as prvalues (7.5.4.3 [expr.prim.id.qual] paragraph 2; 7.6.1.5 [expr.ref] bullet 6.3.2; and 7.6.4 [expr.mptr.oper] paragraph 6). It would simplify the specification if such expressions were categorized as lvalues. (See also [this pull request](#).)

Notes from the August, 2020 teleconference:

CWG preferred that the unbound case (i.e., $\&x : f$) should be an lvalue, while the bound case should be a prvalue.

Proposed resolution (April, 2021):

1. Change 7.5.4.3 [expr.prim.id.qual] paragraph 5, converting the running text into a bulleted list, as follows:

The result of a *qualified-id* Q is the entity it denotes (6.5.5 [basic.lookup.qual]). The type of the expression is the type of the result. The result is an lvalue if the member is

- a function **other than a non-static member function,**
- **a non-static member function if Q is the operand of a unary $\&$ operator,**
- a variable,
- a structured binding (9.6 [dcl.struct.bind]), **or**
- ~~a static member function, or~~
- a data member,

and a prvalue otherwise.

2. Change 7.6.2.2 [expr.unary.op] paragraph 3 as follows:

~~The result of the~~ **The operand of the** unary $\&$ operator **shall be an lvalue of some type τ .**
The result is a ~~pointer to its operand~~ **prvalue.**

- If the operand is a *qualified-id* naming a non-static or variant member m of some class C ~~with type T ,~~ the result has type “pointer to member of class C of type τ ” and ~~is a prvalue designating~~ **designates** $C : m$.
- Otherwise, ~~if the operand is an lvalue of type T , the resulting expression is a prvalue of~~ **result has** type “pointer to τ ” ~~whose result is a pointer~~ **and points** to the designated object (6.7.1 [intro.memory]) or function (**6.8.3 [basic.compound]**). [Note 2: In particular, taking the address of a variable of type “cv τ ” yields a pointer of type “pointer to cv τ ”. —end note]
- ~~Otherwise, the program is ill-formed.~~

[Drafting note: neither 7.6.1.5 [expr.ref] bullet 6.3.2,

- Otherwise (when E_2 refers to a non-static member function), $E_1.E_2$ is a prvalue. The expression can be used only as the left-hand operand of a member function call (11.4.2

[class.mfct]). [Note 5: Any redundant set of parentheses surrounding the expression is ignored (7.5.3 [expr.prim.paren]). —end note]

nor 7.6.4 [expr.mptr.oper] paragraph 6,

...The result of a `.*` expression whose second operand is a pointer to a member function is a prvalue...

requires any change.]

2465. Coroutine parameters passed to a promise constructor

Section: 9.5.4 [dcl.fct.def.coroutine] **Status:** ready **Submitter:** Gor Nishanov **Date:** 2020-10-19

The resolution of issue 2436 (in [P2107R0](#)) deleted the sentence

A reference to a parameter in the *function-body* of the coroutine and in the call to the coroutine promise constructor is replaced by a reference to its copy.

replacing it with new wording in 7.5.4.2 [expr.prim.id.unqual] paragraph 1:

An *identifier* that names a coroutine parameter refers to the copy of the parameter (9.5.4 [dcl.fct.def.coroutine]).

This new approach no longer covers coroutine parameters passed to a promise constructor, since the constructor call is implicit, as described in 7.5.4.2 [expr.prim.id.unqual] paragraph 5.

Suggested resolution:

1. Change 7.5.4.2 [expr.prim.id.unqual] paragraph 4 as follows:

In the following, p_i is an lvalue of type P_i , where p_1 denotes `*this` and p_{i+1} denotes the i^{th} function parameter for a non-static member function, and p_i denotes the i^{th} function parameter otherwise. **Let q_i be the corresponding parameter copy, as described below.**

2. Change 7.5.4.2 [expr.prim.id.unqual] bullet 5.7 as follows:

A coroutine behaves as if its *function-body* were replaced by...

- ...
- *promise-constructor-arguments* is determined as follows: overload resolution is performed on a promise constructor call created by assembling an argument list with lvalues $pq_1 \dots pq_n$. If a viable constructor is found (12.2.3 [over.match.viable]), then *promise-constructor-arguments* is $(pq_1, \dots, pq_n)$, otherwise *promise-constructor-arguments* is empty.

Proposed resolution (April, 2021):

1. Change 9.5.4 [dcl.fct.def.coroutine] paragraph 4 as follows:

In the following, p_i is an lvalue of type P_i , where p_1 denotes `*this` and p_{i+1} denotes the i^{th} function parameter for a non-static member function, and p_i denotes the i^{th} function parameter otherwise. **For a non-static member function, q_1 is an lvalue that denotes `*this`; any other q_i is an lvalue that denotes the parameter copy corresponding to p_i , as described below.**

2. Change 9.5.4 [dcl.fct.def.coroutine] bullet 5.7 as follows:

A coroutine behaves as if its *function-body* were replaced by: ... where

- ...
- *promise-constructor-arguments* is determined as follows: overload resolution is performed on a promise constructor call created by assembling an argument list with lvalues $p_1 \dots p_n$ $q_1 \dots q_n$. If a viable constructor is found (12.2.3 [over.match.viable]), then *promise-constructor-arguments* is $(p_1, \dots, p_n) (q_1, \dots, q_n)$, otherwise *promise-constructor-arguments* is empty.

2466. co_await should be a single evaluation

Section: 7.6.2.4 [expr.await] **Status:** ready **Submitter:** Gor Nishanov **Date:** 2020-10-19

The description of `co_await` should not permit reordering the subexpressions constituting the evaluation of a `co_await` expression. For example, given

```
auto z = co_await coro + co_await coro;
```

the result may be different from the expected

```
auto x = co_await coro;
auto y = co_await coro;
auto z = x + y;
```

Suggested resolution:

Add the following as a new paragraph following 7.6.2.4 [expr.await] paragraph 5:

With respect to an indeterminately-sequenced function call, the operation of `co_await` is a single evaluation. [Note: Therefore a function call cannot intervene between the subexpressions constituting evaluation of a `co_await` expression. —end note]

[Example 1:...

Proposed resolution, May, 2021:

1. Change 6.9.1 [intro.execution] paragraph 11 as follows:

When ~~calling~~ **invoking** a function (whether or not the function is inline), every **value computation and side effect associated with any** argument expression, ~~or with~~ **and** the postfix expression designating the called function, ~~is~~ **are** sequenced before ~~execution of~~ every expression or statement in the body of the called function. For each function invocation **or**

evaluation of an *await-expression* *F*, for every *each* evaluation *A* that occurs does not occur within *F* and every evaluation *B* that does not occur within *F* but is evaluated on the same thread and as part of the same signal handler (if any), either *A* is sequenced before *B* or *B* is sequenced before *A*. is either sequenced before all evaluations that occur within *F* or sequenced after all evaluations that occur within *F*; [Footnote: In other words, function executions do not interleave with each other. —end footnote] **if *F* invokes or resumes a coroutine (7.6.2.4 [expr.await]), only evaluations subsequent to the previous suspension (if any) and prior to the next suspension (if any) are considered to occur within *F*.** [Note 7: If *A* and *B* would not otherwise be sequenced then they are indeterminately sequenced. —end note]

2. Add the following note at the end of 7.6.2.4 [expr.await] paragraph 5:

The *await-expression* evaluates the (possibly-converted) *o* expression and the *await-ready* expression, then:

- ...

[Note: With respect to sequencing, an *await-expression* is indivisible (6.9.1 [intro.execution]). —end note]

Drafting note: No change is needed in 6.9.1 [intro.execution] paragraph 8:

...An expression *X* is said to be sequenced before an expression *Y* if every value computation and every side effect associated with the expression *X* is sequenced before every value computation and every side effect associated with the expression *Y*.

Additional note, May, 2021:

Note 7 in 6.9.1 [intro.execution] paragraph 11 refers to evaluations *A* and *B*, even though the edit to that paragraph above removes those names. This discrepancy was noticed only after CWG approved the change to the normative wording. Since it involves only the wording of a non-normative note, the problem will be addressed editorially. See [editorial issue 4612](#).

2474. Cv-qualification and deletion

Section: 7.6.2.9 [expr.delete] **Status:** ready **Submitter:** Unknown **Date:** 2020-10-29

(From editorial issue [4305](#).)

According to 7.6.2.9 [expr.delete] paragraph 3,

In a single-object delete expression, if the static type of the object to be deleted is different from its dynamic type and the selected deallocation function (see below) is not a destroying operator delete, the static type shall be a base class of the dynamic type of the object to be deleted and the static type shall have a virtual destructor or the behavior is undefined. In an array delete expression, if the dynamic type of the object to be deleted differs from its static type, the behavior is undefined.

Both the static type and the dynamic type include cv-qualification, and requiring agreement in qualification between the two for deletion is not intended. Perhaps the restriction should be to similar types instead of identical types?

Notes from the December, 2020 teleconference:

“Similar types” raises issues with arrays of unknown bounds, but a change to allow for differences in cv-qualification is needed.

Notes from the May 25, 2021 teleconference:

It was observed that current implementations store the total number of class objects in a multi-dimensional array in a “cookie” in the array allocation overhead, rather than the number of top-level array elements, and thus are able to invoke the destructors correctly even if the type being deleted is an array of unknown bound. Consequently, it was decided that use of the “similar” criterion was appropriate.

Proposed resolution, May, 2021:

Change 7.6.2.9 [expr.delete] paragraph 3 as follows:

In a single-object delete expression, if the static type of the object to be deleted is ~~different from~~ **not similar (7.3.6 [conv.qual]) to** its dynamic type and the selected deallocation function (see below) is not a destroying operator delete, the static type shall be a base class of the dynamic type of the object to be deleted and the static type shall have a virtual destructor or the behavior is undefined. In an array delete expression, if the dynamic type of the object to be deleted ~~differs from~~ **is not similar to** its static type, the behavior is undefined.

2477. Defaulted vs deleted copy constructors/assignment operators

Section: 11.4.5.3 [class.copy.ctor] **Status:** ready **Submitter:** Andrew Rogers **Date:** 2021-02-04

According to 11.4.5.3 [class.copy.ctor] paragraph 6,

If the class definition does not explicitly declare a copy constructor, a non-explicit one is declared *implicitly*. If the class definition declares a move constructor or move assignment operator, the implicitly declared copy constructor is defined as deleted; otherwise, it is defined as defaulted (9.5 [dcl.fct.def]).

However, this rule is contradicted by paragraph 10, which lists a number of other reasons why a defaulted copy constructor will be defined as deleted, rather than being “defined as defaulted,” as required by paragraph 6:

A defaulted copy/move constructor for a class x is defined as deleted (9.5.3 [dcl.fct.def.delete]) if x has:

- ...

A similar contradiction exists for copy assignment operators in 11.4.6 [class.copy.assign] paragraphs 2 and 7.

Proposed resolution (April, 2021):

1. Change 11.4.5.3 [class.copy.ctor] paragraph 6 as follows:

If the class definition does not explicitly declare a copy constructor, a non-explicit one is declared *implicitly*. If the class definition declares a move constructor or move assignment operator, the implicitly declared copy constructor is defined as deleted; otherwise, it is ~~defined~~

as defaulted (9.5 [dcl.fct.def]). The latter case is deprecated if the class has a user-declared copy assignment operator or a user-declared destructor (D.9 [depr.impldec]).

2. Change 11.4.6 [class.copy.assign] paragraph 2 as follows:

If the class definition does not explicitly declare a copy assignment operator, one is declared *implicitly*. If the class definition declares a move constructor or move assignment operator, the implicitly declared copy assignment operator is defined as deleted; otherwise, it is ~~defined as~~ defaulted (9.5 [dcl.fct.def]). The latter case is deprecated if the class has a user-declared copy constructor or a user-declared destructor (D.9). The implicitly-declared...

2479. Missing specifications for `constexpr` and `constexpr`

Section: 6.9.3.1 [basic.start.main] **Status:** ready **Submitter:** Davis Herring **Date:** 2020-10-09

There are several places where the `constexpr` and/or `constexpr` keywords should be mentioned but are not:

6.9.3.1 [basic.start.main] paragraph 3:

A program that defines `main` as deleted or that declares `main` to be `inline`, `static`, or `constexpr` is ill-formed.

9.3.4.1 [dcl.meaning.general] paragraph 2:

A `static`, `thread_local`, `extern`, `mutable`, `friend`, `inline`, `virtual`, `constexpr`, or `typedef` specifier or an *explicit-specifier* applies directly to each *declarator-id* in an *init-declarator-list* or *member-declarator-list*...

11.4.5.1 [class.ctor.general] paragraph 1:

...In a constructor declaration, each *decl-specifier* in the optional *decl-specifier-seq* shall be `friend`, `inline`, `constexpr`, or an *explicit-specifier*.

Proposed resolution, May, 2021:

1. Change 6.9.3.1 [basic.start.main] paragraph 3 as follows:

...A program that defines `main` as deleted or that declares `main` to be `inline`, `static`, ~~or~~ `constexpr`, **`constexpr`** is ill-formed...

2. Change 9.3.4.1 [dcl.meaning.general] paragraph 4 as follows:

A `static`, `thread_local`, `extern`, `mutable`, `friend`, `inline`, `virtual`, `constexpr`, **`constexpr`**, **`constexpr`**, or `typedef` specifier or an *explicit-specifier* applies directly to each *declarator-id* in a declaration; the type specified for each *declarator-id* depends on both the *decl-specifier-seq* and its *declarator*.

3. Change 11.4.5.1 [class.ctor.general] paragraph 5 as follows:

...Constructors do not have names. In a constructor declaration, each *decl-specifier* in the optional *decl-specifier-seq* shall be `friend`, `inline`, `constexpr`, **`constexpr`**, or an *explicit-specifier*.

2481. Cv-qualification of temporary to which a reference is bound

Section: 9.4.4 [dcl.init.ref] **Status:** ready **Submitter:** Jiang An **Date:** 2021-03-20

According to 9.4.4 [dcl.init.ref] bullet 5.4.2, when a reference is initialized with a non-class value and the referenced type is not reference-related to the type of the initializer,

- Otherwise, the initializer expression is implicitly converted to a prvalue of type “cv1 T1”. The temporary materialization conversion is applied and the reference is bound to the result.

According to 7.2.2 [expr.type] paragraph 2, the cv-qualification is discarded before invoking the temporary materialization conversion:

If a prvalue initially has the type “cv T”, where T is a cv-unqualified non-class, non-array type, the type of the expression is adjusted to T prior to any further analysis.

This results in a reference-to-const being bound to a non-const object, meaning that a `const_cast` of the reference to a reference-to-nonconst would allow a well-defined modification of the value:

```
constexpr const int &r = 42;
const_cast<int &>(r) = 23;    // Well-defined
static_assert(r == 42);      // Ill-formed, non-constant expression
```

This was different from the situation before the advent of the temporary materialization conversion in C++17, when the description of the reference binding created the temporary explicitly with the cv-qualified type:

If T1 is a non-class type, a temporary of type “cv1 T1” is created and copy-initialized (8.5) from the initializer expression. The reference is then bound to the temporary.

Presumably this difference was unintentional and should be reverted.

Proposed resolution, May, 2021:

Change 9.4.4 [dcl.init.ref] bullet 5.4.2 as follows:

A reference to type “cv1 T1” is initialized by an expression of type “cv2 T2” as follows:

- ...
- Otherwise:
 - If T1 or T2 is a class type and T1 is not reference-related to T2...
 - Otherwise, the initializer expression is implicitly converted to a prvalue of type “cv1 T1”. The temporary materialization conversion is applied, **considering the type of the prvalue to be “cv1 T1”**, and the reference is bound to the result.